ZIBA PROPERTY · AI DELIVERY PLATFORM

ReleaseGuard AI

One AI platform for every team that ships software: requirements, development, QA, production and business performance, in one place you can talk to.



Muhammad Umer
Senior Full Stack Developer, Ziba Property

Laravel · Vue · FastAPI
Portal and 4 AI microservices

Claude · Gemini · OpenAI
Any approved AI provider

CONTENTS

What's inside

03 **How it was built**
Claude Code, .claude & devcrew

04 **The problem**
Time lost between teams

05 **The solution & architecture**
One portal, four AI services

07 **Voice command centre & assistant**
Feature 1

09 **PRD checker, flows & walkthrough**
Feature 2

12 **Production monitoring & Module E**
Feature 3

13 **Sales performance & command center**
Features 4 and 5

15 **Developer toolkit & how to install it**
Feature 6 · devcrew

17 **QA testing & how to set up the QA crew**
Feature 7

19 **Administration & security**
Connections, services, roles

20 **How each team uses it**
A day with ReleaseGuard

21 **Impact: time saved**
Estimates, per team and per week

23 **Quality evidence**
395 automated tests

24 **Installation**
Up and running in 6 steps

25 **Test plan for the judges**
What to do, what you should see

27 **Status & roadmap**
Done, ready, next



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

HOW IT WAS BUILT

One developer, an AI crew, about 20 hours

```
# the project's own Claude Code set-up
releaseguard-ai/
├── CLAUDE.md      project brief + non-negotiables
├── HANDOFF.md     memory between sessions
└── .claude/
    ├── rules/     8 mandatory standards
    │   core · requirements · architecture
    │   coding · database · security
    │   performance · testing
    ├── commands/ 9 workflows
    │   /start /plan /feature /implement
    │   /fix /review /audit /test /status
    └── agents/    8 specialists
        planner · developer · tester
        requirement-auditor · security-
        reviewer · performance-reviewer
        frontend-developer · 3d-developer
```

Skills, safety hooks and the 3D office come from the **devcrew** plugin (Module A), the same toolkit every team now gets.

1. **Set the ground rules once**
   8 rule files: do only what is asked, ask when unclear, OWASP security, never claim an untested result
2. `/plan` turns each idea into acceptance criteria
   The planner and requirement-auditor agents; questions come back before any code
3. `/implement` and `/feature` build it
   Developer, frontend and 3D agents follow the architecture and coding standards
4. `/fix` reproduces every bug first
   Root cause, smallest fix, a regression test that fails before and passes after
5. `/review`, `/audit`, `/test` check every change
   Security and performance reviewers; 395 tests run for real, results reported honestly
6. **HANDOFF.md** carries the context
   Each new session reads it with `/start` and continues where the last stopped



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

THE PROBLEM

Every team loses time in the gaps between teams

Product & BA

PRDs reach the sprint with missing rules, unclear flows and contradictions that nobody spotted.

Developers

Questions bounce between Dev and QA; every developer uses AI differently, with no shared rules or safety.

QA

Regression is manual and repetitive; test automation needs code most testers don't write.

DevOps & on-call

Alarms arrive without context; disks fill up before anyone notices.

Executives

Sales, delivery and production numbers live in different systems; a status report takes hours.

Sales leads

Agent and agency performance sits in the CRM database, hard to see without a developer.

Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

THE SOLUTION

One portal. Specialist AI services. Every team.

ReleaseGuard connects Jira & Confluence, AWS, Slack and the CRM & HomeNet database, and puts an AI behind each job: checking PRDs, explaining alarms, answering questions and running the developer and QA crew.

Real data only

Every screen shows live data. Demo data is always labelled **DEMO**, and states follow real events, never timers.

Secure by design

Encrypted connections, a token per service, read-only CRM access, and answers scoped to each person's role.

Any AI provider

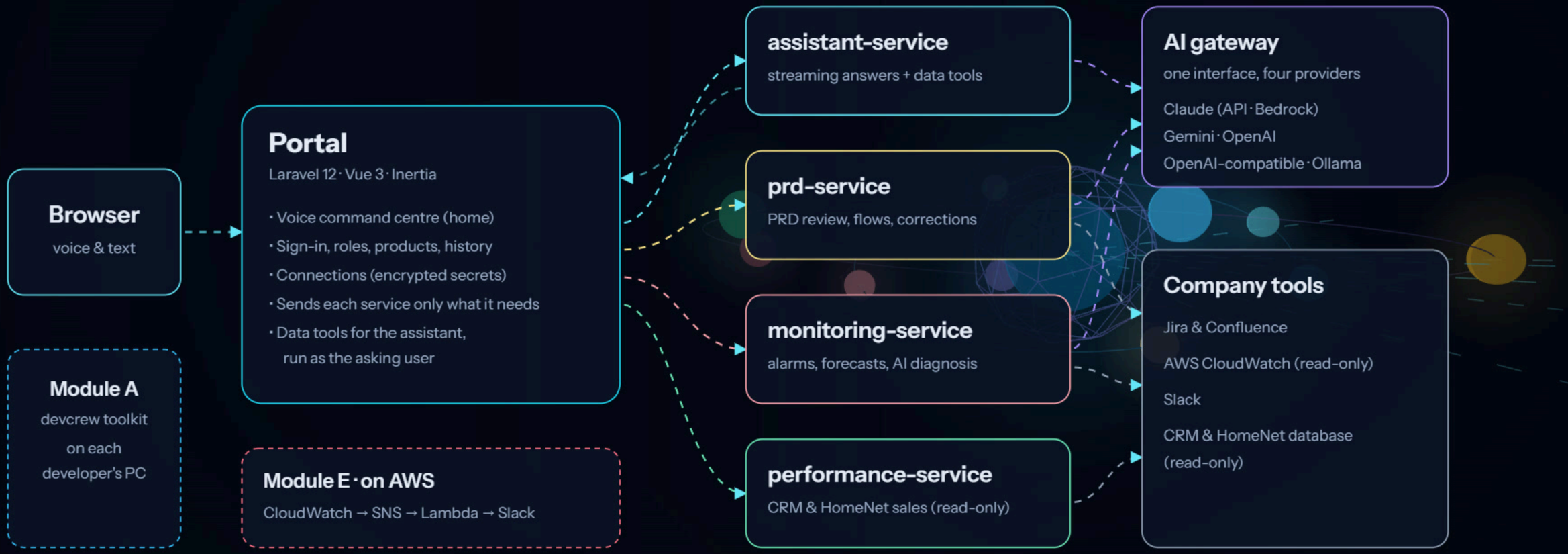
Claude, Gemini, OpenAI or any compatible API (Groq, Mistral, DeepSeek, local Ollama), switched on one page.



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

ARCHITECTURE

A portal that orchestrates microservices



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

FEATURE 1 · VOICE COMMAND CENTRE

The home screen: ask, and see everything live

- Six live widgets: production, portal & services, developer toolkit, sales, PRD readiness and platform activity
- Refreshed from real data every 15 seconds (sales every 60 s); each shows when it last updated
- Tap the orb and speak, or type · Stop listening any time with Esc
- Expand or collapse any widget; the page never scrolls
- One failing source shows its own error, the rest keep working



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

FEATURE 1 · THE ASSISTANT

Answers from company data, never guesses



- 10 read-only data tools: portfolio, monitoring, PRD reviews & flows, alarm diagnoses, team activity, sales, platform status, toolkit
- Tools run **as the asking person** · members see only their products
- Shows which data it checked under each answer
- Real answer: the **5 blocking questions** in the Ziba Property monetization PRD · each traced to a contradiction
- Keeps the conversation: the last 20 messages as context

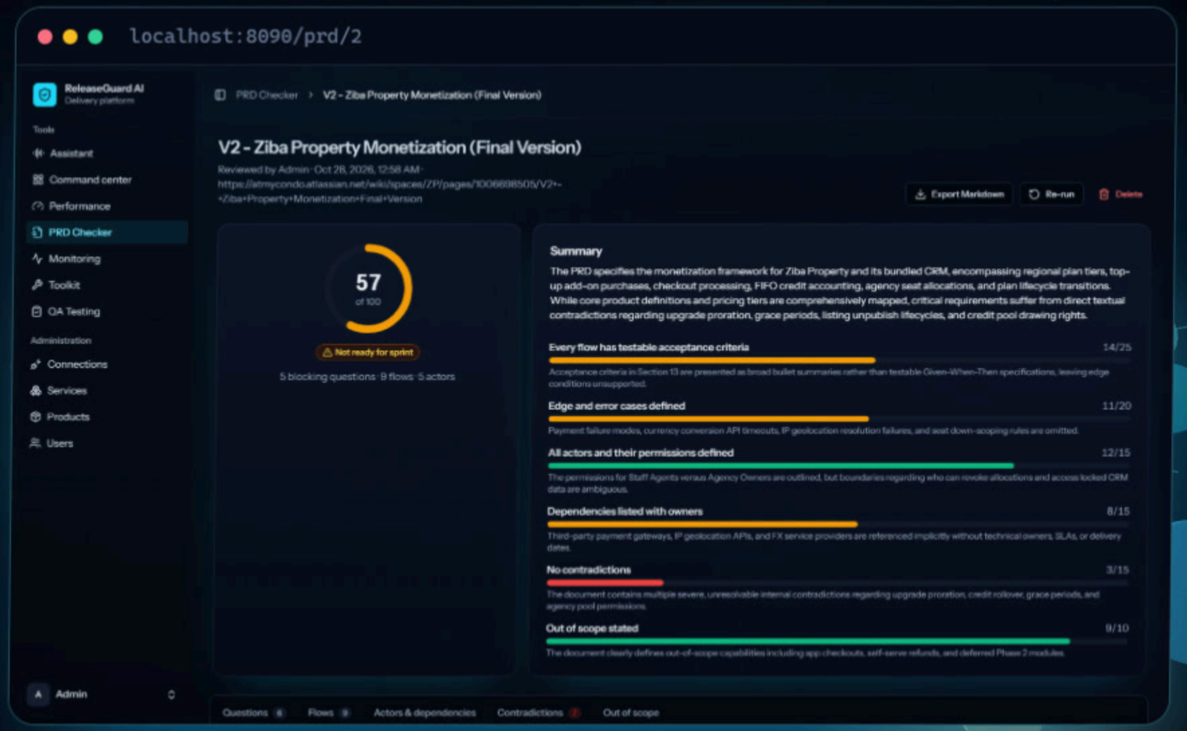


Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

FEATURE 2 · PRD CHECKER

Is this PRD ready for the sprint?

- Upload PDF, Word, Markdown or text, paste it, or link a Confluence page
- Readiness score out of 100 across 6 areas · 70+ enters a sprint
- Blocking questions for the PM, and contradictions quoted word for word
- Actors, permissions and third-party dependencies with owners
- Given/When/Then acceptance criteria, and what the PRD doesn't state
- Export to Markdown for Jira or Confluence

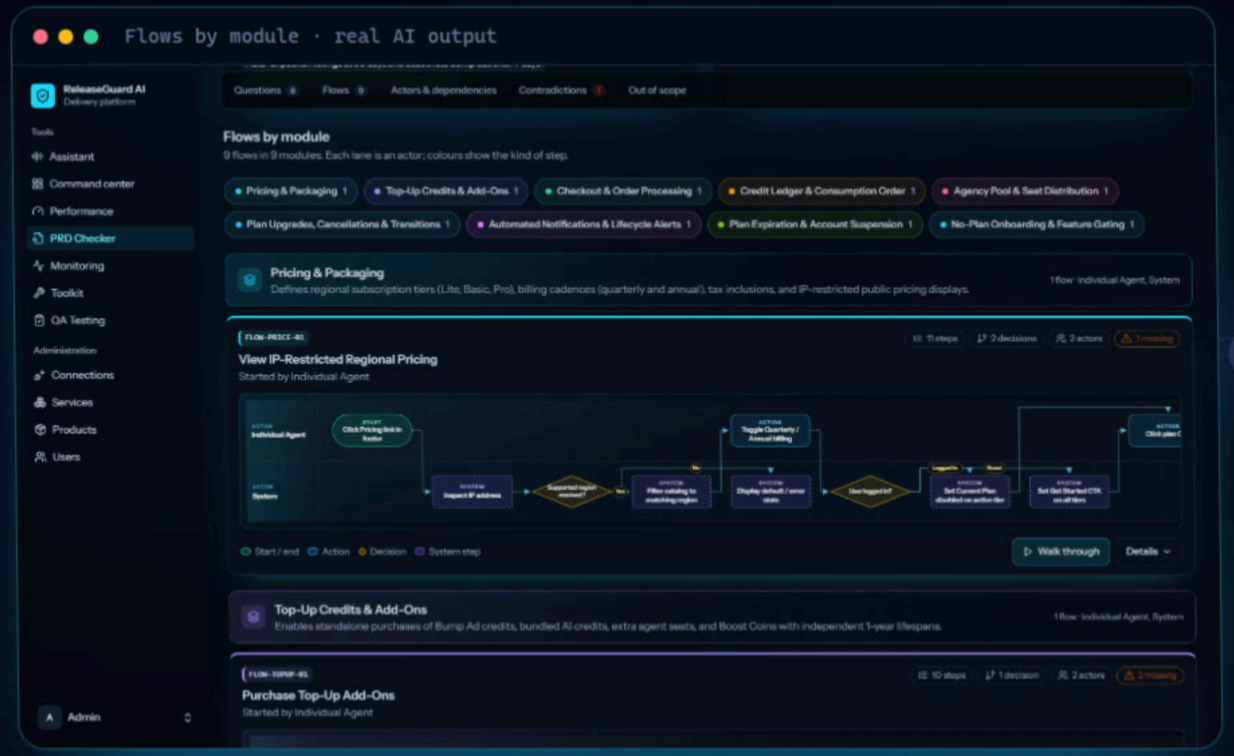




Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

FEATURE 2 · FLOWS BY MODULE

Every module's flow, drawn by actor



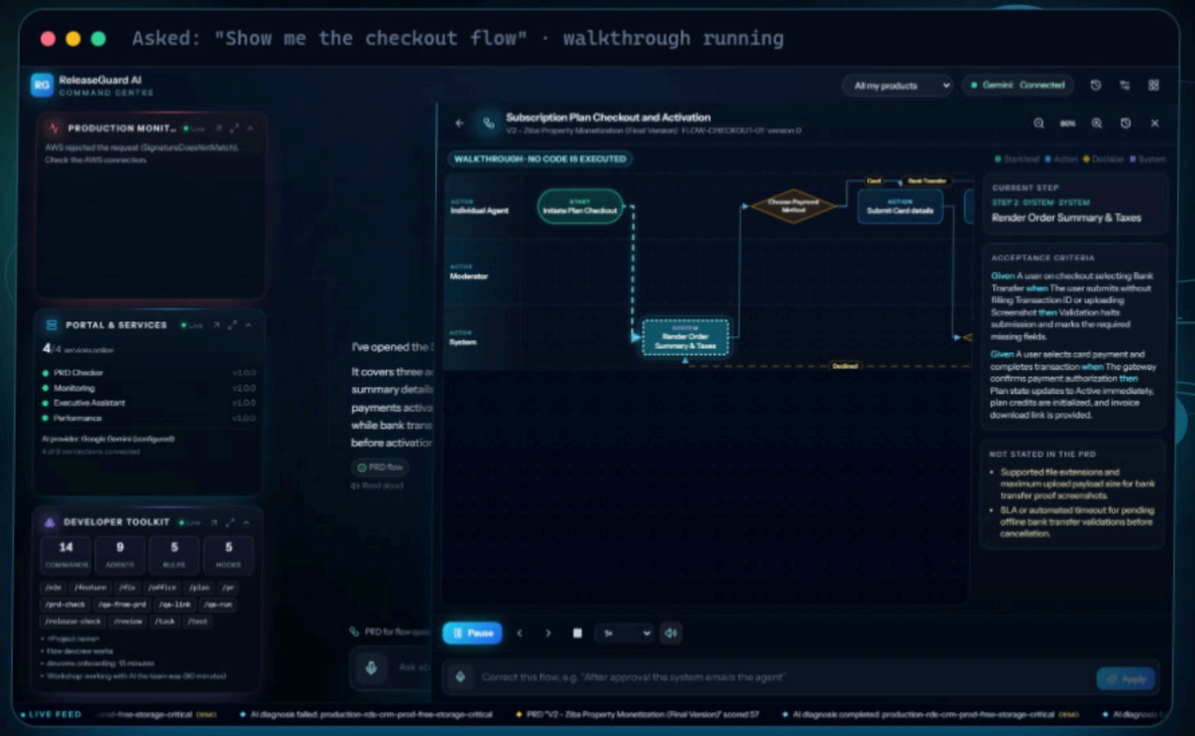
- The AI finds the PRD's modules and writes flows for each one
- Swimlanes: one lane per actor, including the system
- Colours by step: start, action, decision, system step
- Every branch drawn · about 10–25 steps a flow
- Questions link straight to their flow

Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

FEATURE 2 · WALKTHROUGH & CORRECTIONS

Walk through a flow. Fix it by voice.

- **Run walkthrough** lights each step and stops at every decision until you pick a branch
- Narrated aloud if you like · no code is executed
- Say or type a correction: "After approval the system emails the agent"
- The AI redraws the flow and saves it as a **new version**; every version can be restored
- Ask the assistant "show me the checkout flow" and the panel opens



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

FEATURE 3 · PRODUCTION MONITORING

Catch server problems before users do

In the portal

- CloudWatch alarms for database, web servers, load balancer and uptime, refreshed every 10 s
- **AI explain**: likely cause, impact and next steps for any alarm
- Storage forecasts: "database full in 2.8 days"
- Read-only AWS keys; labelled demo data until AWS is connected

Module E · on AWS

- CloudFormation stack: alarms → SNS → incident-assistant Lambda
- The Lambda posts an AI explanation of each alarm to Slack
- 40 automated tests for the Lambda and templates
- BUILT · NOT YET DEPLOYED

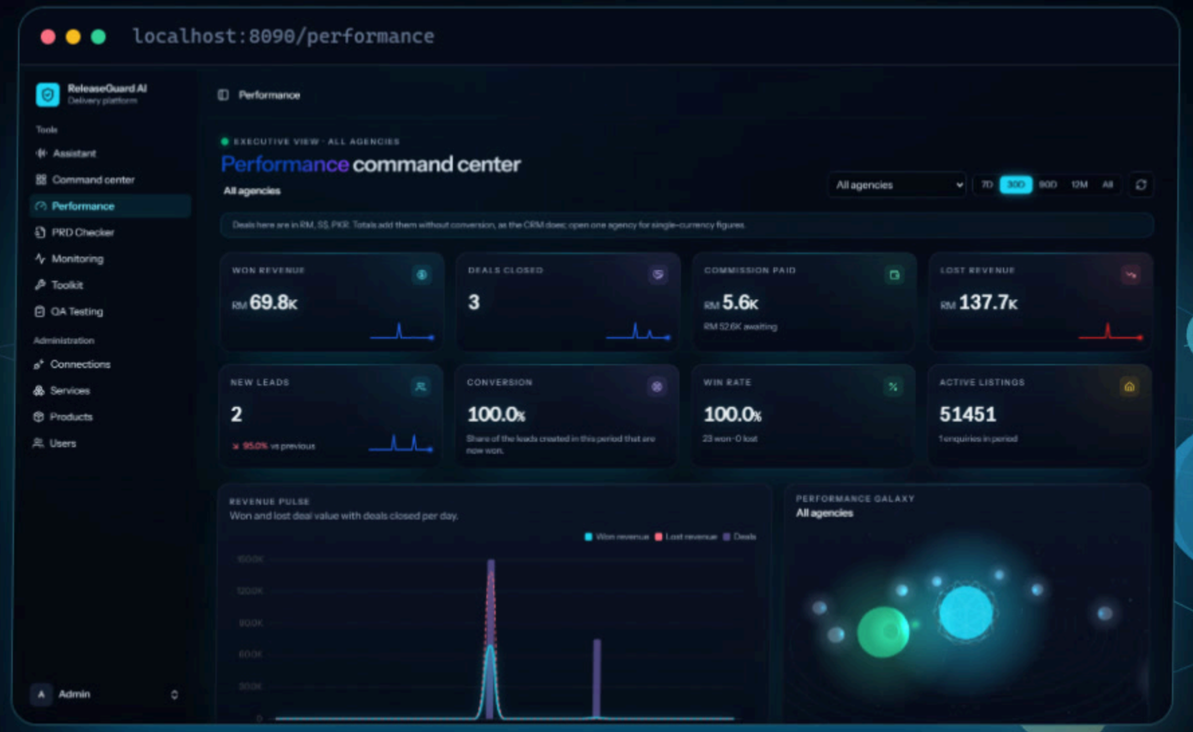


Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

FEATURE 4 · SALES PERFORMANCE

CRM & HomeNet numbers, without a developer

- Won and lost revenue, deals, leads, conversion, commission, listings
- Filter by agency and period: 7 days to all time
- Read-only access to the CRM database
- CRM users sign in with their CRM account and see **only their own scope**, from their live CRM role
- Mixed currencies are flagged, never silently converted



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

FEATURE 5 · COMMAND CENTER

The whole business on one page



- Which integrations are connected, at a glance
- Business pulse for the last 30 days
- Product health: alarms and PRD readiness per product
- One click to ask the assistant, open performance or live monitoring



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

FEATURE 6 · DEVELOPER TOOLKIT (MODULE A)

An AI dev crew, the same for everyone

- `/devcrew:fix ZP-1234` routes the work: plan → build → test → QA → regression → review
- 9 agents, 14 commands, shared team rules and Definition of Done
- Safety hooks block reading `.env` and keys, database wipes and force-push
- A live 3D office shows which agent works on what
- Installs per project for you only; adds nothing to commit



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

FEATURE 6 · INSTALL DEVCREW

From zero to your first crew run in 15 minutes

1. Install **Node.js** and **Git**, then Claude Code
   `npm install -g @anthropic-ai/claude-code`
2. Sign in with **your own company seat**
   Run `claude`, type `/login`; never share a login
3. Get the toolkit
   Toolkit page → **Download toolkit** (or `git clone`)
4. Install it into your project
   Preview with `--dry-run`; nothing is added to the project or to git
5. Connect Jira & Confluence
   In Claude Code: `/mcp` → **atlassian** → sign in
6. Work with the crew
   `/devcrew:fix ZP-1234` · watch it at **localhost:4317**

```
# once per computer
$ npm install -g @anthropic-ai/claude-code
$ claude        # then /login

# once per project (unzipped from the portal)
$ node ai-dev-toolkit/install.js path/to/project --dry-run
$ node ai-dev-toolkit/install.js path/to/project

# every day, in Claude Code
› /devcrew:fix ZP-1234      # bug → plan, fix, test, review
› /devcrew:feature ZP-1300  # new feature, same pipeline
› /devcrew:office           # open the 3D office

# remove · update
$ node ai-dev-toolkit/install.js path/to/project --uninstall
$ git pull       # inside ai-dev-toolkit
```

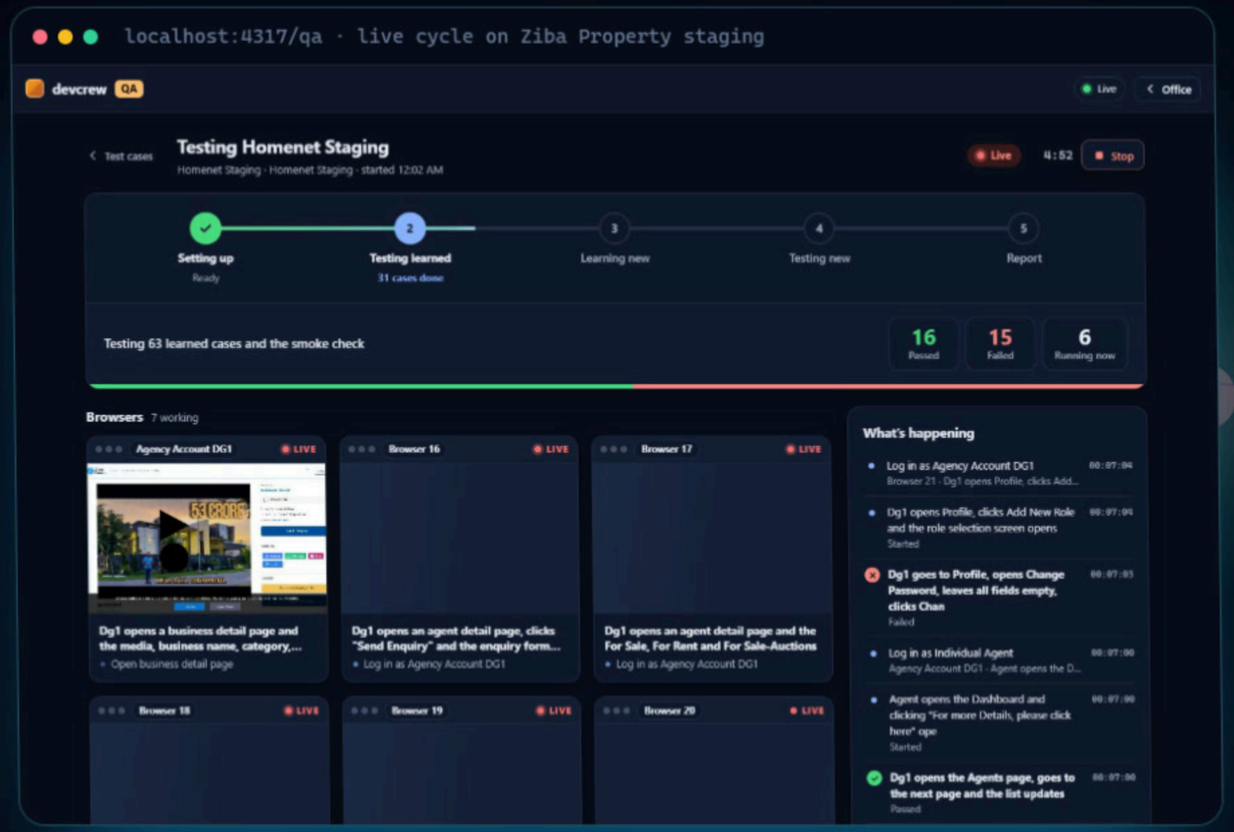
Full guide: `toolkit/docs/ONBOARDING.md` · team workshop: `toolkit/docs/WORKSHOP.md`



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

FEATURE 7 · QA TESTING

Automated QA, in plain English



- Testers write cases the way they would explain them to a colleague
- Learned once by AI, then replayed in seconds with no AI
- Up to 6 real browsers at once, watched live
- Each failure gets a root cause and steps to reproduce
- Cases can be drafted straight from a PRD



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

FEATURE 7 · SET UP THE QA CREW

Testers need no project and no code

For QA testers

1. **Get the toolkit**
   Node.js + Claude Code, signed in with your own seat
2. **Install devcrew into an empty folder**
   `node ai-dev-toolkit/install.js C:\qa-workspace`
3. **Open that folder in Claude Code**
   Type `/devcrew:office`; it runs on your computer only
4. **Open localhost:4317/qa and bookmark it**
   This is where testers work from now on
5. **Add app and a test account**
   Its address per environment (local, staging, UAT)
6. **Paste cases in plain English → Run test cycle**
   "Admin opens the Users page and sees the admin user in the list"

For developers

```
# once, inside your project
› /devcrew:qa-link <app-id>

# fix or build as usual: after the
# regression check the crew runs QA's
# cases on your local app
› /devcrew:fix <JIRA-KEY>

# or run the cycle yourself, any time
# (tests and reports, never changes code)
› /devcrew:qa-run
```

First run installs the test browser once (about 700 MB). Cases are learned once by AI, then replay in seconds with no AI. Failures come with a root cause and steps to reproduce.



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

ADMINISTRATION & SECURITY

Secure by default, simple to run

Connections

Keys are encrypted, never shown again, and sent only to the service that needs them.

Services

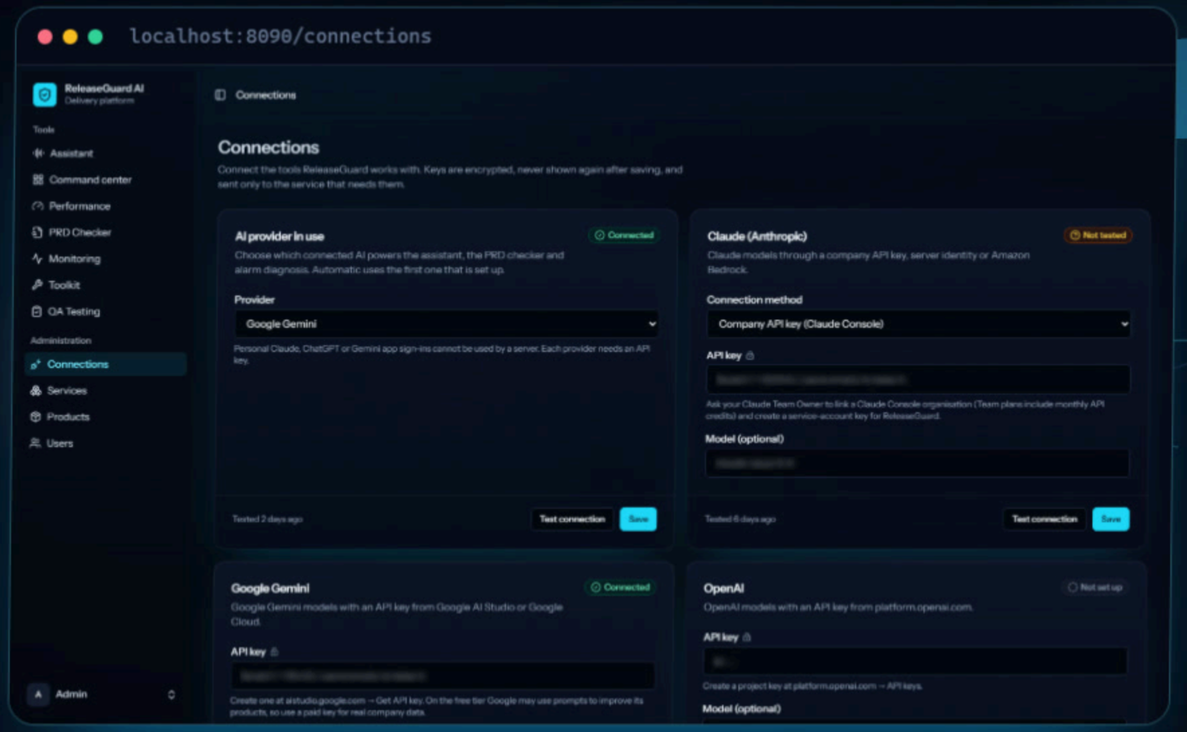
Health, version and capabilities of every service; each needs its own token.

Users & roles

Admins and members; members see only the products they belong to.

Safe AI use

Only approved API credentials, never personal app logins. Rate limits on every AI call.





Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

HOW EACH TEAM USES IT

A day with ReleaseGuard

Product owner / BA

Uploads the PRD, answers the blocking questions, fixes flows by voice, exports the review to Confluence.

Developer

Reads the flow and criteria before coding; runs `/devcrew:fix` so the crew plans, builds, tests and reviews.

QA engineer

Writes cases in plain English, runs them in parallel browsers, files bugs with the root cause attached.

DevOps / on-call

Gets an AI explanation with each alarm in the portal and Slack; watches storage forecasts.

Executive

Opens the command centre and asks: "How did sales go this month? Which PRDs are not ready?"

Sales lead / CRM user

Signs in with the CRM account and sees their own agents' and agencies' performance.

Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

IMPACT

Time saved, team by team

TEAM	TASK	BEFORE	WITH RELEASEGUARD	SAVED
Product / BA	Review a PRD for gaps, criteria and questions	~3 h	~30 min (AI 1–3 min + human check)	**~2.5 h / PRD**
Dev + QA	Agree a feature's flows and acceptance criteria	~2 h	~30 min (flows ready before the sprint)	**~1.5 h / feature**
Developers	Plan, test and self-review a bug fix	~60 min	~20 min (devcrew pipeline)	**~40 min / fix**
QA	Run a 40-case regression cycle	~4 h	~20 min (replay + read the report)	**~3.7 h / cycle**
DevOps	Triage a production alarm	~45 min	~10 min (AI diagnosis)	**~35 min / alarm**
Executives	Compile the weekly sales & delivery status	~2 h	~10 min (live view + ask)	**~1.8 h / week**
New joiners	Set up the team's AI workflow and rules	~4 h	~15 min (onboarding guide)	**~3.7 h / person**

⚠ Estimates, not measurements: typical times for these tasks, to be replaced with the team's own before/after timings.



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

• IMPACT · ONE PRODUCT TEAM, ONE WEEK

About 3 working days back, every week

≈ 23.6 hours / week

≈ 100 hours a month for one product team, using the estimates on the previous slide.

⚠ Illustrative estimate. Assumed weekly workload shown on the right.

ASSUMED EACH WEEK	×	SAVED	HOURS
PRDs reviewed	2	2.5 h	5.0
Features agreed Dev + QA	4	1.5 h	6.0
Bug fixes	8	40 min	5.3
Regression cycles	1	3.7 h	3.7
Production alarms	3	35 min	1.8
Weekly status report	1	1.8 h	1.8
Total			**≈ 23.6**



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

QUALITY EVIDENCE

395 automated tests, all passing

Run on 30 October 2026 before this presentation was made. Every part has its own suite.

144

Portal

Roles, secrets, uploads, services, assistant tools, flows, live widgets

161

AI services & Module E

PRD 43 · Module E 40 · assistant 30 · AI gateway 19 · monitoring 16 · performance 13 (+1 skipped)

90

Developer toolkit

QA testing 72 · 3D office 18



INSTALLATION

Up and running in 6 steps

1. Install **Docker Desktop** and Git
   Port 8080 must be free, or set `PORTAL_PORT` in `.env`

2. Get the code and open its folder
   `cd releaseguard-ai`

3. Create `.env` from `.env.example`
   Fill in `APP_KEY` and the 4 service tokens; the commands are in the file

4. Build and start everything
   `docker compose up --build -d` (the first build takes several minutes)

5. Create your admin account
   It asks for a password (8+ characters)

6. Open **http://localhost:8080** and sign in
   Connections → add an AI key (a free Gemini key works) → Test → Save

```
# 3 · configuration
$ cp .env.example .env
# generate a value (run once per key and token)
$ docker run --rm php:8.3-cli php -r \
   "echo bin2hex(random_bytes(24)), PHP_EOL;"

# 4 · build and start
$ docker compose up --build -d

# 5 · first admin
$ docker compose exec portal php artisan \
   portal:make-admin judge@example.com --name="Judge"
```



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

FOR THE JUDGES · TEST PLAN 1 OF 2

Check it yourself: first 15 minutes

#	DO THIS	YOU SHOULD SEE
1	**Services** page	All 4 services show **Online**
2	Open **Assistant** (the home page)	Six widgets fill with data and show "Live" or "12s ago"; demo data marked **DEMO**
3	Type "Which PRDs are not ready for sprint?"	The answer streams in, with the data it checked listed under it
4	In Chrome, tap the orb and speak; press Esc	"Listening" while you speak; Esc stops listening
5	**PRD Checker** → Paste text → paste `presentation/sample-prd/agent-invitations.md` → Review PRD	In 1–3 minutes: a score out of 100, blocking questions, contradictions and flows by module
6	On a flow: **Walk through**	Steps light up in order and it stops at each decision until you pick a branch



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

FOR THE JUDGES · TEST PLAN 2 OF 2

Roles, monitoring, toolkit and the test suites

#	DO THIS	YOU SHOULD SEE
7	In the flow panel, type a correction and press **Apply**	The flow is redrawn as version 1; **Versions** still has the original
8	Pick the PRD under the composer; ask "show me the invite flow"	The flow panel opens on that flow
9	**Monitoring** → **AI explain** on an alarm	Labelled demo alarms (no AWS needed) and an AI diagnosis
10	**Users** → add a member; sign in as them in a private window	No admin pages; only the products they belong to
11	`node toolkit/plugin/office/start.js`, then `demo.js`	http://localhost:4317 plays a labelled demo run of the 3D office
12	Run the test suites (PHP 8.2+, Python 3.12, Node 20)	Portal `php artisan test` · services `pytest` · toolkit `node --test`: all pass

Full steps with exact commands: `presentation/JUDGES-GUIDE.md`



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

HONEST STATUS

What's done, and what's next

BUILT & TESTED

- Portal and voice command centre
- PRD checker with flows by module (run on a real PRD with Gemini)
- Assistant, monitoring and performance services
- devcrew toolkit, QA testing, 3D office

READY TO DEPLOY

- Module E on AWS (CloudFormation + Lambda)
- Slack alerts for alarms
- Company accounts in place of the local demo login

NEXT

- QA coverage service (Module C)
- Jira delivery KPIs and calendar in the assistant
- A natural neural voice
- Measure the real time saved per team



Developed by **Muhammad Umer** · Senior Full Stack Developer, Ziba Property

THANK YOU

Ship with confidence.

ReleaseGuard AI turns the gaps between teams into one conversation: clear requirements, safer code, faster QA, calmer production and numbers on demand.

PRD ready before the sprint

Dev one AI crew for all

QA plain-English automation

Ops alarms explained

Execs ask anything



Muhammad Umer
Senior Full Stack Developer, Ziba Property

≈ 20 hours
Complete project development time